



DataSynth

Enterprise Synthetic Data Platform

Reference Knowledge Graphs for Audit Analytics & ML

Rust core • 17 modular crates • Group Audit Engine • Audit-Methodology Layer

Executive Overview

Version 5.6.0

Deterministic, privacy-preserving synthetic data generation
for enterprise finance, group audit, compliance testing,
and machine learning.

Companion paper: Ivertowski, M. (2026). *DataSynth: Reference Knowledge Graphs for Enterprise Audit Analytics through Synthetic Data Generation with Provable Statistical Properties*.

SSRN: ssrn.com/abstract=6538639 • In-repo: `paper/main.tex`

Built with Rust • 17 active crates • 200 K+ entries/sec

Open-source (Apache-2.0) • Commercial SDKs at vynfi.com

GitHub: <https://github.com/mivertowski/SyntheticData>

Contents

1	Executive Summary	3
1.1	At a Glance	4
2	Reference Knowledge Graphs	5
2.1	The L4 Audit-Graph Schema	5
2.2	Why Knowledge Graphs	5
3	Enterprise Process Simulation	7
3.1	Process Families	7
3.2	Coherence Engine	7
3.3	Standards Coverage (Single Entity)	8
4	Group Audit Engine	9
4.1	Three-Phase Architecture	9
4.2	Standards Coverage (Group)	10
4.3	Multi-Period Continuity	10
4.4	Output Layout	10
5	Audit-Methodology Layer	11
5.1	What Ships in the Layer	11
5.2	Legacy YAML-Driven FSM (Still Available)	11
5.3	Output Artefacts	12
6	Statistical Foundations	13
6.1	Distributions in the Pipeline	13
6.2	Validation Tests (Always-On)	13
6.3	Industry Profiles	14
7	AI-Augmented Generation	15
7.1	Neural Diffusion	15
7.2	Statistical Diffusion Backend	15
7.3	LLM-Driven Workflows	15
7.4	Auto-Tune Loop	15
7.5	Adversarial Testing	15
7.6	ACFE-Aligned Fraud Taxonomy	15
8	Privacy & Fingerprinting	17
8.1	Workflow	17
8.2	Privacy Levels	17
8.3	Federated Fingerprinting & Certificates	17
8.4	Fidelity Evaluation	17
9	Analytics-Ready Output	18
9.1	The 42-Column <code>journal_entries.csv</code>	18
9.2	Configurable NULL Profiles	18
9.3	Strict COA-Coverage Validation	19
9.4	Output Layout (Per Run)	19
10	Reference Datasets	20

11 Architecture	21
11.1 Layered Crate Architecture	21
11.2 Crate Roles	22
11.3 Production-Grade Infrastructure	23
12 Compliance & Governance	24
12.1 EU AI Act Posture	24
12.2 Regulatory Compliance Documentation	24
12.3 Quality Gates	24
13 Getting Started	25
13.1 Quick Start (Single Entity)	25
13.2 Group Audit Pipeline	25
13.3 Fingerprint-Driven Synthesis	26
13.4 Server Mode	26
13.5 Python Integration	26
13.6 Companion Academic Paper	26

1 Executive Summary

DataSynth is a high-performance synthetic data platform purpose-built for enterprise accounting, group audit, audit analytics, and machine learning. Written in Rust for maximum throughput and memory safety, it generates realistic, internally coherent financial data that satisfies the statistical and structural properties of real-world enterprise resource planning (ERP) systems — and, since v5.0, an entire *multi-entity group consolidation* on top, IFRS / IAS 21 / IAS 28 / IFRS 10 compliant by construction.

Three pillars

- **Enterprise simulation depth** — 14 process families (GL, P2P, O2C, S2C, H2R, manufacturing, financial reporting, tax, treasury, ESG, banking/AML, audit, intercompany, period close), 100+ output tables, generation-time invariants (debits = credits, Assets = Liabilities + Equity, Benford compliance).
- **Multi-entity group audit** — a three-phase manifest / shard / aggregate engine produces standalone single-entity archives *plus* consolidated FS, IC matching, eliminations, IAS 21 translation, NCI / equity-method rollforwards, CGU goodwill impairment testing, and chained multi-period continuity.
- **Methodology-driven audit trail** — the `datasynth-audit-fsm` crate ships a typed audit-methodology layer: Big-4 ISA spine (4 phases, 17 procedures), 4 firm overlays, 7 jurisdictional overlays, 6 KYC/AML workflows, banking-form ontologies, Bayesian RMM scoring, an L4 typed property-graph schema, and SHA-256 Merkle tamper-evidence for working-paper bundles.

Why DataSynth?

- **No real data required** — eliminates privacy, regulatory, and procurement barriers to analytics development.
- **Full auditability** — deterministic, seeded generation; every dataset is bit-for-bit reproducible.
- **ML-ready from day one** — ground-truth labels for fraud, anomalies, data quality issues, and ISA-240 audit flags ship alongside every record.
- **Domain depth** — the full accounting lifecycle, multi-entity consolidation, and complete audit methodology.
- **Empirically grounded** — statistical distributions calibrated against real enterprise data; formal proofs in the companion paper [1].
- **Provenance you can verify** — per-file SHA-256, W3C PROV-JSON lineage graphs, and Merkle-rooted working-paper bundles.

1.1 At a Glance

200 K+ entries/sec (single thread)	17 modular Rust crates	14 enterprise process families
12+7 Big-4 + jurisdictional audit overlays	8 IFRS / IAS standards modelled	60+ ACFE-aligned fraud schemes
6 KYC / AML workflows	4 differential-privacy levels	100+ output tables (CSV / JSON / Parquet / XBRL)
5 industry presets	15 holiday calendar regions	XBRL 2.1 instance export (US GAAP & IFRS)

Companion paper

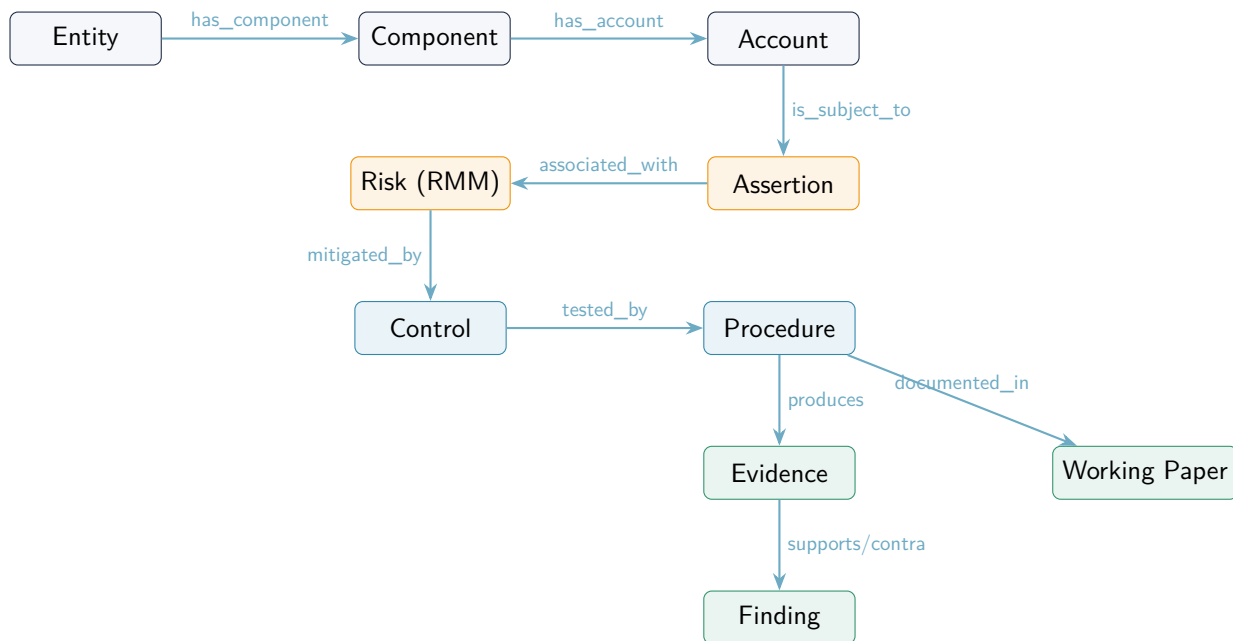
Formal statements of the distributional, copula, and DP-composition properties summarised in this overview — together with proofs and ablation studies — appear in the companion paper [1]. Readers building academic benchmarks or formal evaluations should consult §§5–9 of the paper directly. The LaTeX source is available in-repo at `paper/main.tex`.

2 Reference Knowledge Graphs

DataSynth builds what the companion paper calls a *reference knowledge graph*: a deterministic, content-addressable graph whose nodes encode every domain entity (companies, vendors, customers, accounts, journal entries, controls, procedures, evidence) and whose edges encode every business relationship (transactions, references, ownership, control coverage, assertion linkage, evidence support). Generation is reproducible byte-for-byte from a seed, so the same graph can be regenerated, hashed, signed, and audited.

2.1 The L4 Audit-Graph Schema

The audit-methodology layer ships a **typed property-graph schema** — 11 node types crossed with 10 edge types — that mirrors the assertion- and risk-driven structure of an ISA / PCAOB engagement.



This schema is the same one a Big-4 firm carries in its proprietary audit-management software — only here it is open, content-addressable, and generated end-to-end alongside the underlying ledger. The companion paper formalises it as a typed property graph and proves the soundness of the acceptance-gate predicates that consume it.

2.2 Why Knowledge Graphs

A reference graph turns the audit dataset from a collection of CSVs into a **queryable, verifiable, and benchmarkable artefact**:

- Cross-process queries (vendor → PO → GR → invoice → payment → JE → control → procedure → working paper) are graph traversals, not multi-table SQL joins.
- Provenance is intrinsic — every node carries its generator type, seed, and generation timestamp; every edge carries its derivation rule.
- Anomalies travel through the graph: when a fraudulent vendor invoice is injected, the propagation path through document flows, JE postings, and control failures is observable.

- ML benchmarks become reproducible — the same seed yields the same graph, so model A and model B can be compared on the *exact* same data without re-randomisation.

3 Enterprise Process Simulation

Every process chain produces cross-referenced master data, documents, and journal entries that respect double-entry bookkeeping, document chain integrity, and balance sheet equation invariants *at generation time*.

3.1 Process Families

Process family	Scope
General Ledger	Journal entries, chart of accounts (small / medium / large), ACDOCA, balanced postings, FX revaluation
Procure-to-Pay	POs, goods receipts, vendor invoices, payments, three-way match with configurable tolerances
Order-to-Cash	Sales orders, deliveries, customer invoices, receipts, dunning, AR aging
Source-to-Contract	Spend analysis, sourcing projects, supplier qualification, RFx, bids, contracts, scorecards
Hire-to-Retire	Payroll, time & attendance, expenses, benefits, defined-benefit pensions, stock-based compensation
Manufacturing	Production orders, BOMs, WIP costing, quality inspections, cycle counts, inventory movements
Financial Reporting	BS / IS / CF, statement of changes in equity, KPIs, budgets, segment reporting, notes, XBRL
Tax	Multi-jurisdiction VAT / GST, ASC 740 / IAS 12 provisions, deferred tax rollforward, withholding tax, uncertain tax positions
Treasury	Cash positioning, forecasts, pooling, hedging (ASC 815 / IFRS 9), debt covenants, derivatives
ESG	GHG Scope 1/2/3, energy / water / waste, workforce diversity, pay equity, GRI / SASB / TCFD disclosures
Banking / AML	20 AML typologies, criminal networks, velocity features, KYC tiers, SAR narratives
Audit	ISA lifecycle, ISA 600 group audit, SOX 302/404, methodology blueprints
Intercompany	IC matching (multi-strategy), transfer pricing methods, eliminations, currency translation, NCI
Period Close	Depreciation runs, accruals, year-end closing, tax provisions, FX revaluation

3.2 Coherence Engine

A cross-module coherence layer connects every record to every other record:

- **Document chains** — PO → GR → Invoice → Payment with proper reference fields and audit-trail flags.
- **Vendor/customer networks** — 3-tier supply chain (Tier1 / 2 / 3), 4 cluster types (reliable strategic / standard / transactional / problematic), configurable concentration risk, and bilateral relationships.

- **Customer segmentation** — 4 value segments with prescribed revenue/customer share, life-cycle stages, referral networks, corporate hierarchies.
- **Relationship strength** — weighted composite of transaction volume (log scale), count (sqrt scale), duration, recency (90-day exponential decay), and mutual connections (Jaccard index).
- **Cross-process links** — inventory ↔ P2P/O2C, payment ↔ bank reconciliation, intercompany bilateral matching.

3.3 Standards Coverage (Single Entity)

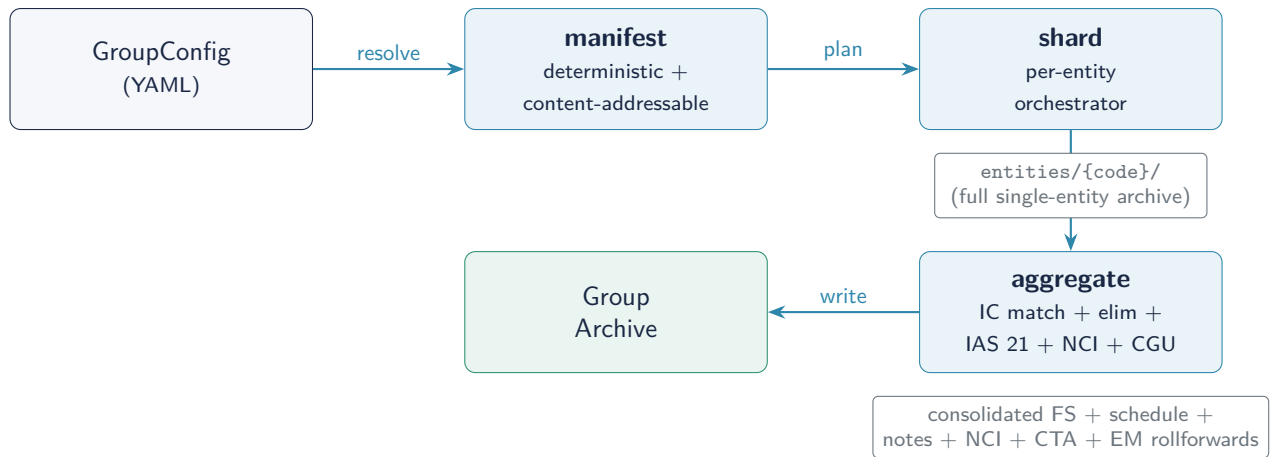
Single-entity output adheres to the following standards by construction:

Domain	Standard & artefacts
Revenue	ASC 606 / IFRS 15: customer contracts, performance obligations, revenue allocation, contract modifications
Leases	ASC 842 / IFRS 16: ROU assets, lease liabilities, finance / operating split
Fair Value	ASC 820 / IFRS 13: Level 1/2/3 hierarchy, valuation methodology disclosures
Impairment	ASC 360 / IAS 36: triggering events, recoverable-amount tests, no-reversal rule
Business Combinations	IFRS 3 / ASC 805: PPA, goodwill, fair-value adjustments, contingent consideration, bargain purchase
ECL Provisions	IFRS 9: 12-month / lifetime ECL stages, provision matrix, ECL movements
Provisions	IAS 37 / ASC 450: provision liabilities, contingent liabilities, unwind of discount
Pensions	IAS 19 / ASC 715: defined-benefit obligation, plan assets, OCI remeasurements
Stock Compensation	ASC 718: grant-date fair value, vesting, APIC stock-comp
Internal Controls (SOX)	SOX 302 certifications, SOX 404 assessments, deficiency matrix, material-weakness classification
ISA Audit Lifecycle	34 ISA standards mapped, analytical procedures (ISA 520), confirmations (ISA 505), opinions (ISA 700/705/706/701)
Audit Data Exchange	ISO 21378:2019 (Audit Data Collection): 3-level account classification (Type / Class / Sub-Class) populated on every CoA account and every JE row, with descriptive names alongside the ISO codes

4 Group Audit Engine

Since v5.0, DataSynth ships a **multi-entity group audit engine** layered above the single-entity pipeline. Per-entity generation stays byte-identical to the standalone flow; the consolidation logic lives entirely in the `datasynth-group` crate. The pipeline is a deterministic **three-phase model**.

4.1 Three-Phase Architecture



1. **manifest** resolves a **GroupConfig** into a deterministic, content-addressable **GroupManifest** — entities, periods, ownership graph, IC pair plan, shard plan, FX / chart-of-accounts masters, audit engagement plan, tax group plan.
2. **shard** drives the orchestrator for one shard of entities and writes a full single-entity archive per entity under `entities/{code}/`.
3. **aggregate** reads shard outputs and runs IC matching, eliminations, IAS 21 currency translation, NCI rollforward, equity-method investments (with suppressed-loss tracking), CGU goodwill-impairment testing, and writes consolidated FS, schedule, notes, and rollforwards.

The CLI auto-detects whether an input config is a **GroupConfig** (top-level `presentation_currency` + `ownership` keys) and dispatches to the group pipeline transparently — existing single-entity configs continue to work unchanged.

4.2 Standards Coverage (Group)

Standard	What is modelled
IAS 21 §39 / §42(b)	Closing-rate translation, CTA rollforward; closing-rate-for-all-items in hyperinflationary economies
IAS 28 §22 / §38	Equity-method investments; suppressed-loss tracking with recovery-against-future-profits
IAS 29 §12	Indexed restatement before IAS 21 closing-rate translation
IAS 36 §10 / 80 / 104 / 124	CGU definition, acquisition-date goodwill allocation, impairment-loss allocation, no-reversal rule
IFRS 3 §19 / §42	Acquisition-date NCI measurement (full vs partial goodwill); mid-period ControlGained re-measurement at fair value
IFRS 5 / ASC 810	NCI presented separately from controlling-interest equity
IFRS 8.13 / ASC 280-10-50-41	Operating-segment disclosures derived from the ownership graph
IFRS 10.23 / IFRS 10.B97	Equity-transaction adjustment for within-control ownership changes; deconsolidation on control loss

4.3 Multi-Period Continuity

The `group generate-chain` subcommand stitches engagements together across periods. Opening balances, NCI, CTA, and equity-method carryforwards flow forward without caller plumbing; the chain helps handle emergent-fuzzy IC matching tolerances, mid-period acquisitions and disposals, and step-acquisition mid-period control-gained transitions (IFRS 3 §42 P&L re-measurement and time-weighted profit attribution).

4.4 Output Layout

Per-engagement archive

```
./group_archive/
manifest.json           # canonical group manifest
entities/
  NESTLE_SA/           (full single-entity archive per shard)
  NESTLE_USA/           ...
consolidated/
  consolidated_financial_statements.json
  consolidation_schedule.json
  notes_to_consolidated_fs.json
  nci_rollforward.json
  cta_rollforward.json
  translation_worksheet.json
  equity_method_investments.json
  equity_method_suppressed_losses.json
  cgu_impairment_tests.json
ic_eliminations/ic_matching_coverage.json
shard_summary.json
```

5 Audit-Methodology Layer

The `datasynth-audit-fsm` crate (v5.5.0) ships a typed audit-methodology layer derived from the [AuditMethodology](#) companion repository. All blueprints, overlays, ontologies, and form schemas are embedded via `include_str!` so loaders are zero-I/O at runtime. The L4 graph schema introduced in §2 is the integration surface.

5.1 What Ships in the Layer

Module	What it carries
<code>big4_methodology</code>	ISA-derived common spine (4 phases, 17 procedures) plus 4 firm overlays inheriting from the spine (EY GAM, PwC Aura, KPMG Clara, Deloitte Omnia) with per-procedure firm tools and signoff chains, plus a cross-firm equivalence map for harmonisation reporting.
<code>jurisdictional_overlay</code>	7 overlays — PCAOB, EU CSRD, UK FRC, ASIC (Australia), JFSA (Japan), ACRA (Singapore), HKICPA (Hong Kong) — 39 procedures total, AND-logic resolution on (<code>jurisdiction</code> × <code>registrant_type</code>).
<code>methodology_blueprint</code>	ISA 600 (Revised, December 2022) and CSRD limited-assurance methodology blueprints.
<code>kyc_blueprint</code>	6 KYC / AML workflows — private banking, correspondent banking, crypto-CASP, periodic KYC, SAR escalation, sanctions-hit remediation.
<code>banking_forms</code>	7 banking form ontologies (MROS SAR, 4 UBS forms, Wolfsberg CBDDQ, FCCQ) and a 228-entry cross-form evidence index unifying fields under canonical terms.
<code>scenario_library</code>	15 deterministic engagement scenarios with expected outcomes (opinion type, going-concern conclusion, EOM paragraph, acceptance gate) for FSM verification.
<code>rmm_scoring</code>	Bayesian Risk-of-Material-Misstatement scoring — 12-factor taxonomy (7 inherent + 5 control), conjugate Beta-Bernoulli updates, $RMM = IR \times CR$ aggregation.
<code>l4_graph</code>	Typed property-graph schema, 11 node types × 10 edge types (see §2).
<code>working_paper_merkle</code>	SHA-256 Merkle tree with replay-protected inclusion proofs; ISA 230 working-paper / engagement-bundle types; canonical <code>bundle_root</code> ordered by <code>wp_id</code> .

5.2 Legacy YAML-Driven FSM (Still Available)

The earlier YAML-driven FSM engine (10 built-in blueprints — FSA, IA, KPMG, PwC, Deloitte, EY GAM, SOC 2, PCAOB, Regulatory) remains available alongside the new modules. Existing integrations continue to work unchanged; new work is recommended on the typed audit-methodology layer for type safety, zero-I/O startup, and Merkle tamper-evidence.

5.3 Output Artefacts

For each engagement the audit generators produce: engagement letters, combined risk assessments (ISA 315), materiality calculations (ISA 320), sampling plans + sampled items (ISA 530), accounting estimates (ISA 540), analytical procedures (ISA 520), confirmations (ISA 505), going-concern assessments (ISA 570), subsequent events (ISA 560), service-organization SOC reports (ISA 402), component-auditor reports (ISA 600), audit opinions + key audit matters (ISA 700/705/706/701), SOX 302 / 404 documentation, ISA / PCAOB cross-reference mappings, and SHA-256 Merkle bundle roots.

6 Statistical Foundations

DataSynth amounts, frequencies, correlations, and temporal patterns follow empirical distributions calibrated against real enterprise data. This section sketches the model; **formal statements and proofs appear in the companion paper, §§5–9 [1]**.

6.1 Distributions in the Pipeline

Family	Use
Log-normal mixtures	Amount sampling — routine / significant / major components with weighted mixing
Benford’s Law	First-digit law compliance with measurable MAD; supports first-digit, first-two-digit, and deviation-injection variants
Gaussian copulas	Cross-field dependency modelling (amount $\times$ line items $\times$ approval level) with rank-preserving inverse-CDF sampling
Clayton / Gumbel / Frank / Student-t	Tail-asymmetric dependence; lower-tail (Clayton) for fraud co-incidence, upper-tail (Gumbel) for shared peak loads
Pareto	Heavy-tailed amounts (vendor concentration, customer revenue)
Weibull	Time-to-event modelling (payment delays, dunning escalation)
Beta	Proportions and percentages (tax rates, allocation ratios)
Zero-inflated	Cost centres, line items where many records carry no value
Conditional	Breakpoint-based generation: amount conditional on document type, vendor tier, etc.
Drift / regime change	Economic cycles, recession depth, growth phases over multi-year horizons
Temporal	Seasonality, holiday calendars (15 regions), business-day conventions, T+N settlement, period-end clustering

6.2 Validation Tests (Always-On)

- **Benford first-digit MAD** — threshold 0.015 (significant deviation by Nigrini’s classification).
- **Distribution fit** — Anderson-Darling and chi-squared against the configured target, $\alpha = 0.05$.
- **Correlation check** — Spearman rank correlation against the configured copula, $\alpha = 0.05$.
- **Balance equation** — assets = liabilities + equity, every JE balanced, every period.

6.3 Industry Profiles

Five built-in industry profiles — Retail, Manufacturing, Financial Services, Healthcare, Technology — pre-configure distributions, vendor counts, customer segmentation, fraud rates, and process emphasis. Three complexity tiers (Small / Medium / Large) drive chart-of-accounts sizing and transaction volume.

7 AI-Augmented Generation

7.1 Neural Diffusion

A Candle-powered score network (DDPM) trained end-to-end on tabular feature slices, with deterministic sampling. GPU acceleration via the `neural-cuda` feature flag with a graceful CPU fallback. Use cases: high-fidelity replication of correlated continuous attributes where copulas are too parametric.

7.2 Statistical Diffusion Backend

Always-on `DiffusionBackend` for denoising and enhancement of sampled-then-perturbed records — improves long-tail recall on rare combinations without invoking neural training.

7.3 LLM-Driven Workflows

Three layers, each gated by the `llm` feature flag:

- **Config generation from natural language** — `datasynth init -from-description "12 months of mid-market retail data with fraud and SOX controls"` produces a complete YAML config. Supports OpenAI, Anthropic, OpenRouter.
- **Template enrichment** — offline deterministic CLI expands vendor / customer / material / employee pools via any OpenAI-compatible endpoint; cached YAML, byte-identical reruns.
- **Audit-finding enrichment** — `FindingLlmEnricher` adds prose recommendations, severity narratives, and remediation steps to generated findings.

7.4 Auto-Tune Loop

`generate` → `evaluate` → AI patch → `regenerate`. The auto-tuner inspects evaluation gaps (Benford deviation, correlation drift, distribution fit failure) and emits a `ConfigPatch` that the orchestrator merges into the config for the next iteration. Bounded by `-max-iterations` (default 3).

7.5 Adversarial Testing

ONNX-loaded fraud-detection or anomaly-detection models can be probed at the boundary via the `adversarial` feature flag (`ort` runtime). The framework generates targeted perturbations to surface decision-boundary weaknesses — useful for hardening models trained on synthetic data before real-data deployment.

7.6 ACFE-Aligned Fraud Taxonomy

60+ fraud schemes mapped to the ACFE Fraud Tree:

- **Fraud:** `FictitiousTransaction`, `RevenueManipulation`, `SplitTransaction`, `RoundTripping`, `GhostEmployee`, `DuplicatePayment`, `BillAndHold`, `ChannelStuffing`, `RelatedPartyDiversion`.
- **Error:** `DuplicateEntry`, `ReversedAmount`, `WrongPeriod`, `WrongAccount`, `MissingReference`.
- **Process:** `LatePosting`, `SkippedApproval`, `ThresholdManipulation`.
- **Statistical:** `UnusualAmount`, `TrendBreak`, `BenfordViolation`.
- **Relational:** `CircularTransaction`, `DormantAccountActivity`.

Every fraud-flagged record passes through a behavioural-bias filter: weekend posting, round-dollar bias, off-hours timestamps, post-close posting — all configurable, all measured by smoke tests on every CI run.

8 Privacy & Fingerprinting

The fingerprint subsystem extracts statistical structure from real data without retaining record-level information, then synthesises new data that matches the captured structure. This is the recommended bridge from a real production dataset to a defensible synthetic benchmark.

8.1 Workflow

1. **Extract** — `datasynth fingerprint extract` reads CSV / Parquet / JSON / a directory and writes a `.dsf` file carrying: column distributions, copula parameters, conditional breakpoints, temporal patterns, cardinalities, NULL rates. Original rows never leave the host.
2. **Validate** — `fingerprint validate` checks the `.dsf` for internal consistency.
3. **Synthesise** — `generate -fingerprint thesis.dsf -scale 1.0` draws a synthetic dataset matching the captured distribution.
4. **Evaluate** — `fingerprint evaluate` computes fidelity metrics (per-column KS / chi-squared, copula recovery, ML utility on a downstream task).

8.2 Privacy Levels

Four progressive privacy levels selectable via `-privacy-level`:

Level	Guarantees
minimal	Aggregate-only statistics; no per-record information retained.
standard	k -anonymity ($k = 5$ default) on quasi-identifier groups; suppression of singleton records.
strong	(ϵ, δ) -differential privacy with $\epsilon = 1.0$, $\delta = 10^{-5}$; calibrated Laplace / Gaussian noise.
paranoid	$\epsilon = 0.1$ DP; per-column suppression of low-density tails; randomised response on categorical attributes.

8.3 Federated Fingerprinting & Certificates

Multiple parties can each extract fingerprints locally, sign them, and exchange only the `.dsf` files. The central aggregator merges distributions under a privacy budget and emits a tamper-evident **certificate** that records the contributing parties, their ϵ allocations, and the SHA-256 root of the merged fingerprint. Formal DP composition (sequential and parallel) is proved in the companion paper [1].

8.4 Fidelity Evaluation

The evaluator returns per-column KS / chi-squared, copula recovery (Frobenius distance on the rank correlation matrix), Benford MAD, balance-equation pass rate, and downstream-task utility (train on synthetic, score on real — TRTS / TSTR). Quality gates can be configured at lenient, default, or strict thresholds; `-quality-gate strict` fails the run on the first violation.

9 Analytics-Ready Output

v5.5.1 widened the default analytics surface based on production-data column profiles reported by external users (notably a graph-neural-network anomaly-detection thesis); v5.6.0 extends it with the ISO 21378 (ADC) account-classification hierarchy. The aim is for downstream Pandas / Polars / Spark notebooks to need zero ETL.

9.1 The 42-Column `journal_entries.csv`

The flat CSV writer emits 42 columns covering identifiers, dates, currency, audit / ETL provenance, ISO-21378 account classification, and analytics-ready derivations:

Group	Columns
Identifiers	<code>document_id</code> , <code>line_number</code> , <code>transaction_id</code> (UUID v5 of (<code>document_id</code> , <code>line_number</code>), deterministic across regenerations)
Period	<code>fiscal_year</code> , <code>fiscal_period</code> , <code>posting_date</code> , <code>document_date</code> , <code>value_date</code>
Document	<code>company_code</code> , <code>document_type</code> , <code>currency</code> , <code>exchange_rate</code> , <code>reference</code> , <code>header_text</code>
Amounts	<code>debit_amount</code> , <code>credit_amount</code> , <code>local_amount</code>
GL	<code>gl_account</code> , <code>account_description</code> (joined from CoA), <code>financial_statement_category</code> (asset / liability / equity / revenue / cogs / operating_expense / other_income_expense / tax / suspense; derived from <code>gl_account</code> prefix)
ISO 21378 (ADC)	<code>account_class</code> (Level-2 code, e.g. A.B for Trade Receivables), <code>account_class_name</code> (descriptive), <code>account_sub_class</code> (Level-3 code, e.g. A.B.A), <code>account_sub_class_name</code> (descriptive). Joined from the CoA at write time.
Cost / profit	<code>cost_center</code> , <code>profit_center</code> , <code>assignment</code>
Provenance	<code>created_by</code> , <code>source</code> (enum), <code>source_system</code> (28-value ERP-realistic taxonomy: SAP-FI/AP, SAP-MM/IV, SAP-SD/ORD, Treasury/CM, Interface/EDI, manual/adjustment, spreadsheet/upload, ...), <code>business_process</code> , <code>ledger</code>
Audit flags	<code>is_fraud</code> , <code>is_anomaly</code> , <code>is_manual</code> (ISA 240), <code>is_post_close</code> (ISA 240)
Tax	<code>tax_code</code>
French GAAP (FEC)	<code>auxiliary_account_number</code> , <code>auxiliary_account_label</code> , <code>lettrage</code> , <code>lettrage_date</code>
Line text	<code>line_text</code>

9.2 Configurable NULL Profiles

`data_quality.missing_values.field_rates` accepts a per-field NULL rate (MCAR / MAR / MNAR / Systematic) so users can match a real production column profile (e.g. `cost_center` 96.5% NULL, `invoice_reference` 100% NULL). Audit-critical identifiers — `document_id`, `gl_account`, `transaction_id`, `posting_date` — are force-protected regardless of user configuration.

9.3 Strict COA-Coverage Validation

The `-validate-coa-coverage` CLI flag (and `PhaseConfig::validate_coa_coverage_strict`) hard-fails the run when any generated JE references a `gl_account` that is not in the chart of accounts. Off by default — a soft warning is logged. Pairs naturally with the always-on COA invariant test.

9.4 Output Layout (Per Run)

Generated files are organised by domain directory under `output/`: `master_data/`, `document_flows/`, `sourcing/`, `hr/`, `manufacturing/`, `subledger/`, `balance/`, `financial_reporting/`, `period_close/`, `intercompany/`, `fx/`, `tax/`, `treasury/`, `project_accounting/`, `esg/`, `accounting_standards/`, `internal_controls/`, `banking/`, `process_mining/`, `audit/`, `labels/`, `graphs/`, `events/`, `standards/`.

10 Reference Datasets

Pre-generated reference datasets are published at huggingface.co/VynFi under Apache 2.0, fully synthetic, zero PII. All datasets are reproducible from the `config.yaml` files shipped in `configs/examples/`.

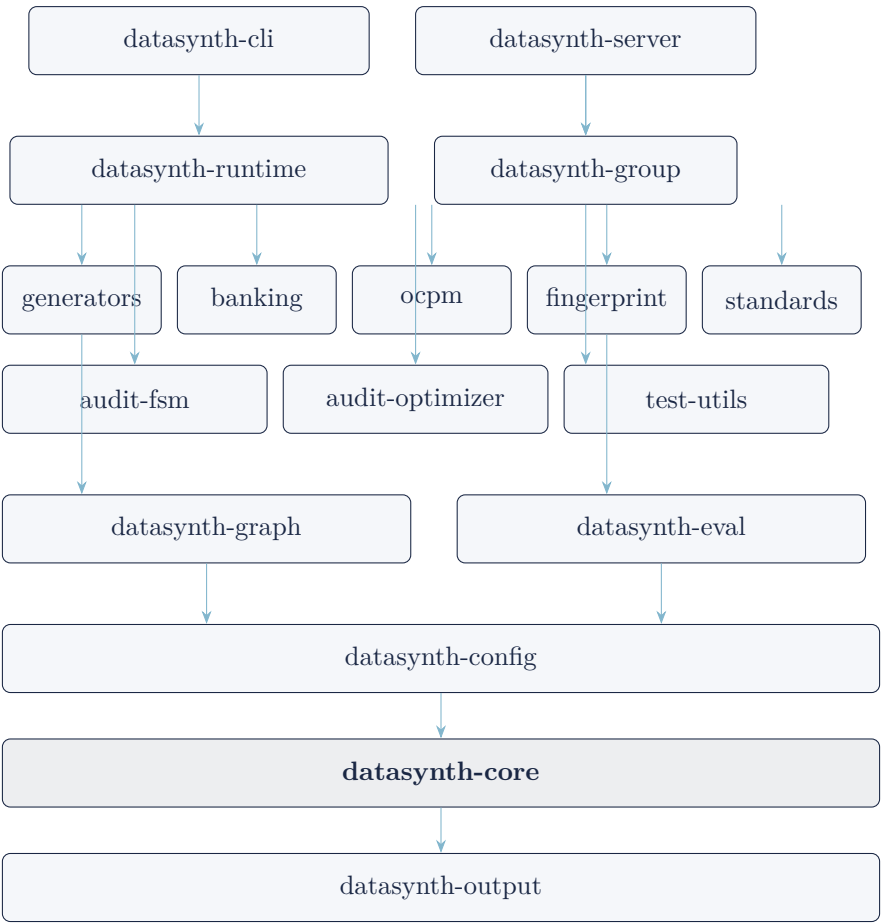
Dataset	Scale	Description
<code>vynfi-group-audit-enterprise-2000</code>	2 000 entities	Multinational ACME holding, 4 functional currencies, 4 759 IC pairs (91.6% matched), full IFRS-compliant consolidated FS + schedule + notes + CTA + NCI + equity-method rollforwards.
<code>vynfi-journal-entries-1m</code>	2.1 M lines	Manufacturing-sector denormalised JE table with 6.92% fraud rate, ISA 240 manual flag, GL chart of accounts.
<code>vynfi-aml-100k</code>	749 K rows	Banking transactions with AML labels, 14 velocity features, 59 columns.
<code>vynfi-audit-p2p</code>	234 docs	P2P document chain (PO / GR / VI / Payment) with fraud labels.
<code>vynfi-ocel-manufacturing</code>	344 events	OCEL 2.0 event log for process mining (pm4py, Celonis).
<code>vynfi-supply-chain-ocel</code>	—	Supply-chain OCEL event log for cross-process mining.
<code>vynfi-sar-narratives</code>	—	Suspicious-activity-report narratives for AML training.

Loading from Python

```
from datasets import load_dataset
ds = load_dataset("VynFi/vynfi-aml-100k", split="train")
df = ds.to_pandas()
```

11 Architecture

11.1 Layered Crate Architecture



11.2 Crate Roles

Crate	Role
<code>datasynt-cli</code>	Binary — generate, validate, init, info, fingerprint, templates, group, scenario, optimizer
<code>datasynt-server</code>	REST + gRPC + WebSocket server with API-key + JWT auth, RBAC, rate limiting
<code>datasynt-runtime</code>	EnhancedOrchestrator coordinating ~30 generation phases
<code>datasynt-group</code>	Multi-entity manifest / shard / aggregate engine (v5.0+)
<code>datasynt-generators</code>	Domain generators (JE, document flows, subledgers, anomalies, audit)
<code>datasynt-banking</code>	KYC / AML banking with fraud typologies
<code>datasynt-ocpm</code>	OCEL 2.0 process mining
<code>datasynt-fingerprint</code>	Privacy-preserving fingerprint extract / synthesis
<code>datasynt-standards</code>	Accounting / audit standards (IFRS, US GAAP, French GAAP, German GAAP, ISA, SOX, PCAOB)
<code>datasynt-graph</code>	Graph export (PyTorch Geometric, Neo4j, DGL, RustGraph)
<code>datasynt-eval</code>	Evaluation framework with auto-tuning
<code>datasynt-config</code>	Configuration schema, validation, presets
<code>datasynt-core</code>	Domain models, traits, distributions, resource guards, templates
<code>datasynt-output</code>	Output sinks (CSV, JSON, NDJSON, Parquet, XBRL, FEC, GoBD, SAP, Oracle, NetSuite)
<code>datasynt-test-utils</code>	Test fixtures
<code>datasynt-audit-fsm</code>	Typed audit-methodology layer (v5.5.0): Big-4 spine, jurisdictional overlays, KYC, banking forms, RMM, L4 graph, Merkle
<code>datasynt-audit-optimizer</code>	Risk-scoping / portfolio / Monte-Carlo / conformance analytics

11.3 Production-Grade Infrastructure

Capability	Details
Deterministic Output	ChaCha8 PRNG with configurable seed; identical seed $\Rightarrow$ identical dataset.
Financial Precision	<code>rust_decimal</code> throughout; no IEEE 754 floating-point artefacts; decimals serialised as strings.
Resource Guards	Unified CPU, memory, and disk monitoring with automatic throttling and graceful degradation (Normal $\rightarrow$ Reduced $\rightarrow$ Minimal $\rightarrow$ Emergency).
Collision-Free IDs	FNV-1a hash-based UUID factory with generator-type discriminators prevents document-ID collisions across parallel generators. UUID v5 for stable per-line transaction IDs.
Streaming Output	CSV, JSON, NDJSON, Apache Parquet (Zstd compression), and XBRL 2.1 instances (US GAAP & IFRS taxonomies). Streaming sinks with backpressure (Block, DropOldest, DropNewest, Buffer).
API Layer	REST + gRPC + WebSocket with Argon2id API-key auth, JWT/OIDC bearer tokens (RS256, Keycloak / Auth0 / Entra ID), RBAC (Admin / Operator / Viewer), sliding-window rate limiting (in-memory or Redis), TLS via rustls.
Plugin SDK	<code>GeneratorPlugin</code> , <code>SinkPlugin</code> , <code>TransformPlugin</code> traits with thread-safe registry.
Webhooks	Fire-and-forget event dispatch (RunStarted / Completed / Failed / GateViolation) with HMAC signing and configurable retry.
Data Lineage	Per-file SHA-256 checksums, <code>LineageGraph</code> tracking config $\rightarrow$ generator $\rightarrow$ output relationships, W3C PROV-JSON export, CLI <code>verify</code> .
COA Coverage Invariant	Always-on test asserting every generated JE <code>gl_account</code> exists in the chart of accounts; strict-mode CLI flag fails the run on orphans.

12 Compliance & Governance

12.1 EU AI Act Posture

DataSynth is a synthetic-data generator, not a high-risk AI system: outputs are synthesised data points, not decisions about natural persons. The platform nonetheless ships features useful for compliant downstream use:

- Generator name, version, timestamp, seed, and config hash machine-readable in every output run.
- Per-file SHA-256 checksums and W3C PROV-JSON lineage for reproducibility audits.
- Differential-privacy budgets emitted as machine-readable certificates when fingerprinting is used.
- Ground-truth labels (fraud, anomaly, data-quality issue, ISA 240 manual flag) for fairness and bias evaluation of downstream models.

12.2 Regulatory Compliance Documentation

Generated audit artefacts include the metadata required by the relevant standard frameworks (ISA / PCAOB / SOX / SOC 2 / CSRD), with cross-reference tables and jurisdiction profiles for cross-border engagements. Working-paper bundles are SHA-256 Merkle-rooted with replay-protected inclusion proofs (ISA 230 working-paper management).

12.3 Quality Gates

Three quality-gate profiles — `lenient`, `default`, `strict` — combine balance-equation, distribution-fit, copula-recovery, Benford-MAD, completeness, and coherence checks. The auto-tune loop and the CI pipeline both consume the same gate definitions.

13 Getting Started

13.1 Quick Start (Single Entity)

CLI

```
git clone https://github.com/mivertowski/SyntheticData.git && cd SyntheticData
cargo build --release

# Demo --- generates a complete dataset with defaults
./target/release/datasynth-data generate --demo --output ./output

# Initialise + generate from a config
./target/release/datasynth-data init --industry manufacturing \
                                     --complexity medium \
                                     -o config.yaml
./target/release/datasynth-data validate --config config.yaml
./target/release/datasynth-data generate --config config.yaml \
                                     --output ./output \
                                     --validate-coa-coverage
```

13.2 Group Audit Pipeline

Multi-entity consolidation

```
# Single-period
datasynth-data group generate \
  --config configs/examples/group/mini_nestle.yaml \
  --out ./group_archive

# Multi-period (chained continuity)
datasynth-data group generate-chain \
  --config configs/examples/group/mini_nestle.yaml \
  --periods 4 \
  --out ./group_archive_q1_q4

# Or use the explicit three-phase pipeline
datasynth-data group manifest --config group.yaml --out manifest.json
datasynth-data group shard --manifest manifest.json --shard-id S_SIG_0001 \
  --out ./shards/S_SIG_0001/
datasynth-data group aggregate --manifest manifest.json --shards-dir ./shards \
  --out ./group_archive
```

13.3 Fingerprint-Driven Synthesis

From real data to defensible synthetic

```
# 1. Extract a privacy-preserving fingerprint from real data
datasynth-data fingerprint extract \
  --input ./real_journals.csv \
  --output ./real.dsf \
  --privacy-level standard

# 2. Validate the .dsf file
datasynth-data fingerprint validate ./real.dsf

# 3. Generate matched synthetic data
datasynth-data generate --fingerprint ./real.dsf \
  --scale 1.0 \
  --output ./synthetic

# 4. Evaluate fidelity
datasynth-data fingerprint evaluate \
  --fingerprint ./real.dsf \
  --synthetic ./synthetic/
```

13.4 Server Mode

REST / gRPC / WebSocket

```
cargo run -p datasynth-server -- --port 3000 --worker-threads 4

# REST: /api/config, /api/stream/{start|stop|pause|resume},
#       /api/stream/trigger/{pattern}
# WebSocket: /ws/events
# Auth: X-API-Key header (Argon2id) or Bearer JWT
```

13.5 Python Integration

The open-source `datasynth-py` wrapper has been retired (v4.4.3). Recommended paths:

- **Commercial SDKs** — official typed Python and TypeScript SDKs at vynfi.com.
- **Subprocess + dataframes** — invoke the `datasynth-data` CLI from Python via `subprocess` and read the generated CSV / JSON / Parquet outputs with `pandas` / `polars` / `pyarrow`.
- **HuggingFace** — skip generation and pull a pre-built reference dataset from huggingface.co/VynFi.

13.6 Companion Academic Paper

The formal treatment of distributional properties, copula construction, DP composition, and the reference-knowledge-graph framework appears in:

Citation

Ivertowski, M. (2026). *DataSynth: Reference Knowledge Graphs for Enterprise Audit Analytics through Synthetic Data Generation with Provable Statistical Properties*. SSRN Working Paper. <https://ssrn.com/abstract=6538639>
In-repo source: `paper/main.tex`.

References

- [1] Michael Ivertowski. DataSynth: Reference knowledge graphs for enterprise audit analytics through synthetic data generation with provable statistical properties. SSRN Working Paper, April 2026. URL <https://ssrn.com/abstract=6538639>. Available at SSRN: <https://ssrn.com/abstract=6538639>. In-repo source: `paper/main.tex`.